

Parallel and Distributed Systems Project 4

Parallelization Techniques on Genetic Algorithm for Acceleration and Performance Increase

Sotirios Karantzoulis AEM:10952
Github (<https://github.com/sot-kara/PDS-prj4>)

September 2026

1 Introduction

In this project, parallelization techniques were used on Optimization Techniques class's final project to achieve speedup and better results. The project consists of a genetic algorithm that trains gaussian functions from a population to predict the form of a 2-variable function.

Namely the true function is $f(u_1, u_2) = \sin(u_1 + u_2) \sin(u_2^2)$ and the gaussians are of the form $G(u_1, u_2) = e^{-\left(\frac{(u_1 - c_1)^2}{2\sigma_1^2} + \frac{(u_2 - c_2)^2}{2\sigma_2^2}\right)}$. At first each gaussian is initialized with random values and a set of 5 is used to make the prediction each time. That set of 5 is the "genome" of a model (**pop**).

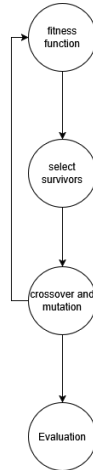


Figure 1: Basic flow of the algorithm

At figure 1 the algorithm structure is apparent. The pops are assessed from a fitness function which was the mean squared error (MSE). Their survival to the next generation is proportional to their performance (aka **Roulette Wheel** method) which is the inverse of the MSE. If a pop is in the top performance it is guaranteed to pass on the next gen (aka **Elitism**). Afterwards, pops breed and exchange "genomes" (i.e. half of their gaussian functions) while there is a chance of their offspring "mutating" which randomizes some gaussian values.

After this whole process repeats for a number of iterations called generations. Finally, the best performing pop of the generation is chosen as the final model from evaluating it with new random points in the fitness function.

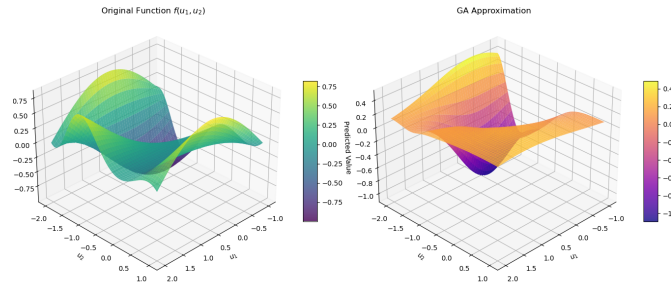


Figure 2: Real vs Predicted function using the Sequential Algorithm

As a reference the sequential algorithm executes in around 4 seconds and has a final MSE that varies around 0.02.

2 Parallelization with OpenMP

It is easy to notice that each loop of assessing each pop, selecting survivors and crossover and mutation can happen entirely in parallel (while keeping the order of operations). So OpenMP was chosen as an easy way to completely parallelize the corresponding for loops that'll be handled by OpenMP workers.

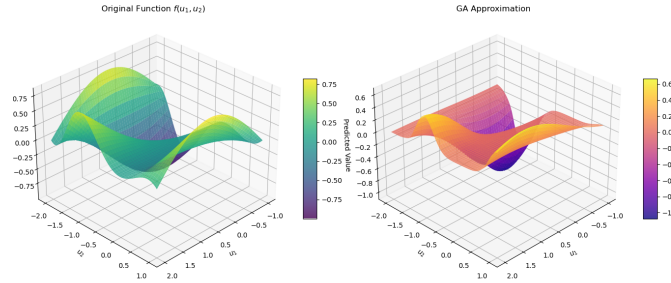


Figure 3: Real vs predicted function using OpenMP

The parallelized version seems to yield similar results in terms of accuracy as the sequential one. On the other hand, it appears to be a lot faster in terms of computing time. Changing the `OMP_NUM_THREADS` environment variable enables benchmarking the OpenMP version to investigate scaling and speedup.



Figure 4: OpenMP Scaling with number of threads used

From the above graph it becomes evident that increasing the number of threads (thus decreasing the load for each thread) dramatically drops the execution time. However, after 12th thread added, the performance is worse due to contesting for CPU time, while also having diminishing returns after that. The best time was at 10 threads at 0.472528 seconds achieving a total speedup of:

$$s = \frac{3.546091}{0.472528} \approx 7.5 = 750\%$$

3 Parallelization with Pthreads: the Island Method

To investigate an interesting approach to this problem, pthreads were used to implement the Island method. In this version the population is divided equally to a number of islands where evolution takes place for a number of epochs independently. After a number of iterations the best of the each island are migrated to their neighboring islands, replacing the worst performers. In the end the best model is chosen from all the islands. This way the genome pool of each island gets periodically refreshed promoting a better possible solution to the problem (in a way increasing "biodiversity").

For the purposes of this project, each island is assigned one pthread thread that handles the genetic algorithm for a number of epochs with a fixed amount of pops (25 in this case). After that the islands exchange migrants by accessing a global array in a thread safe way and then wait to proceed to the next epochs.

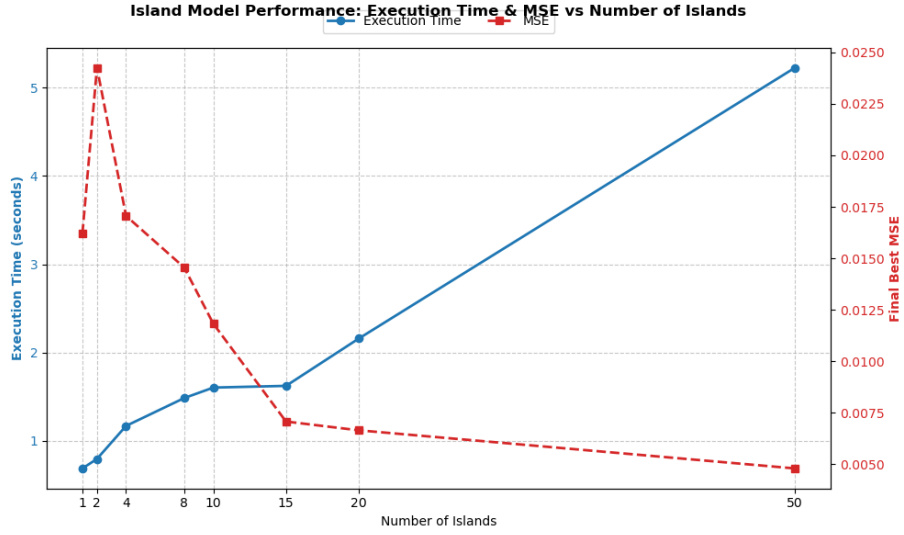


Figure 5: MSE and Execution time vs the number of islands

The above figure makes it clear that increasing the number of islands and, therefore, the global population size, improves the model's accuracy (low MSE). Of course the execution time increases the more islands there are, however it only reaches around 5s. At 50 islands for this specific run there are 1250 pops in total, which would take the sequential algorithm a substantial amount of computation time to evolve until the end.

So this method not only has better results but is comparatively faster than running the sequential algorithm.

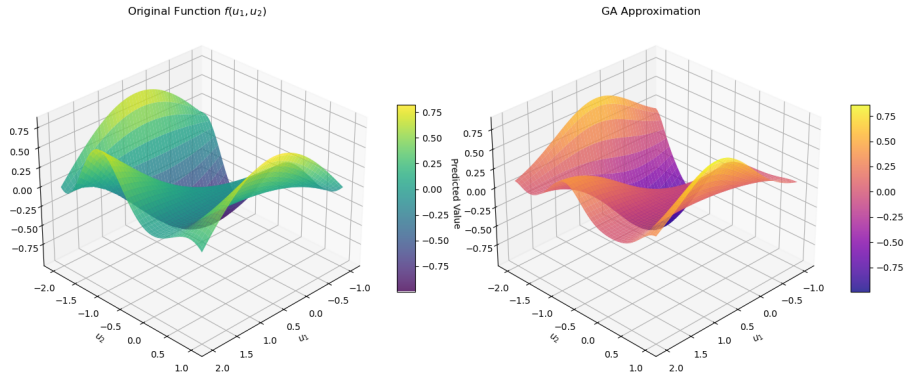


Figure 6: Best model produced by the Island Method. The predicted function is very similar to the true one.